Detecting Transient Protein Filaments in Living Cells

Academic Research Track

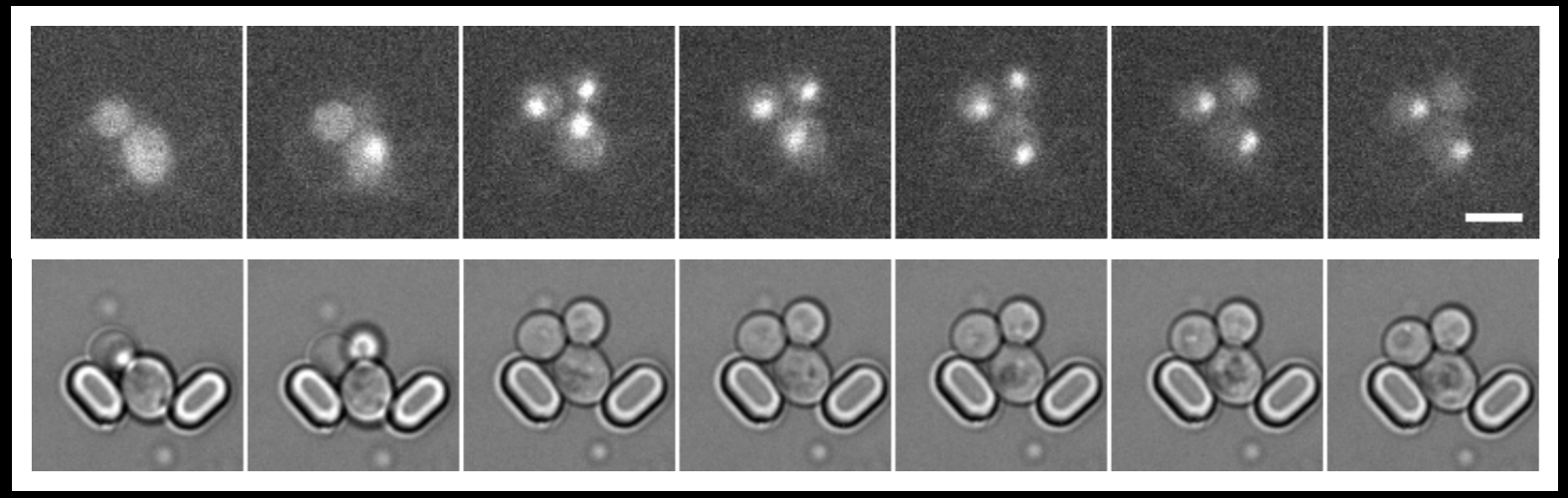
Francois El-Daher · francois.el-daher@ed.ac.uk

BioHackathon Edinburgh 2026 · 20 March

Cells remodel themselves in minutes.

Under stress, proteins reorganise
into new structures within minutes.

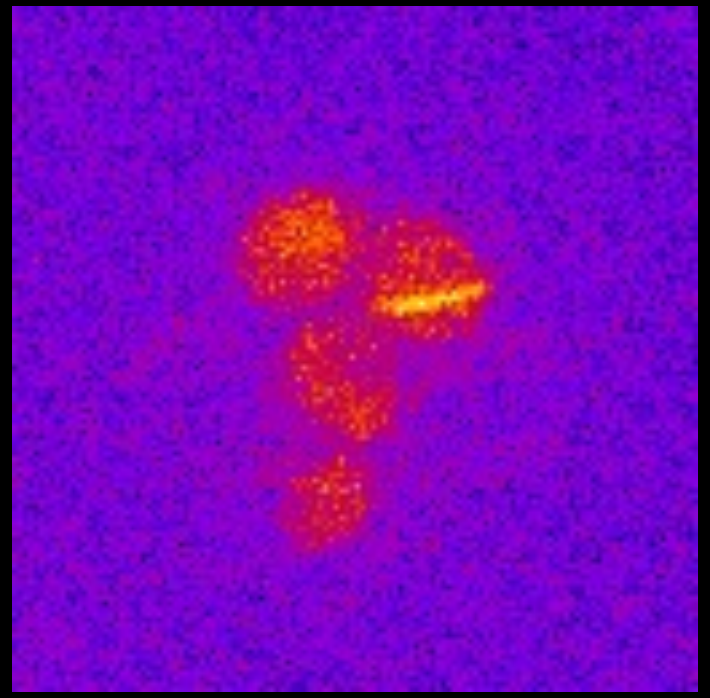
These structures influence how well or badly they survive



time

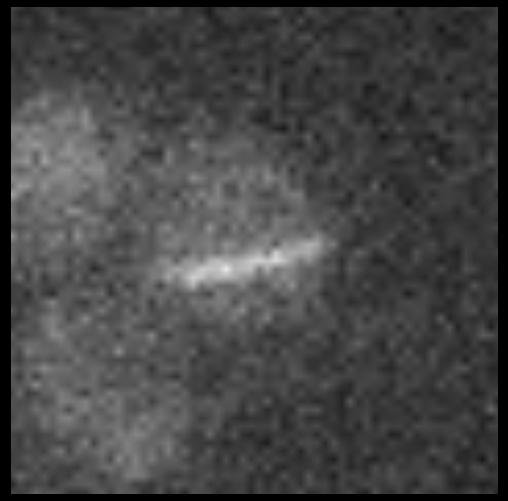
URA7: a protein that forms filaments under stress

- URA7 is essential for synthesis of RNA, DNA and phospholipids
- Under stress → forms cytoophidia (elongated intracellular filaments)
- When assembled: cytoplasm goes dark
- Formation is transient and reversible — timing encodes the stress response
- 15–89% of cells aggregate depending on conditions (high cell-cell heterogeneity)

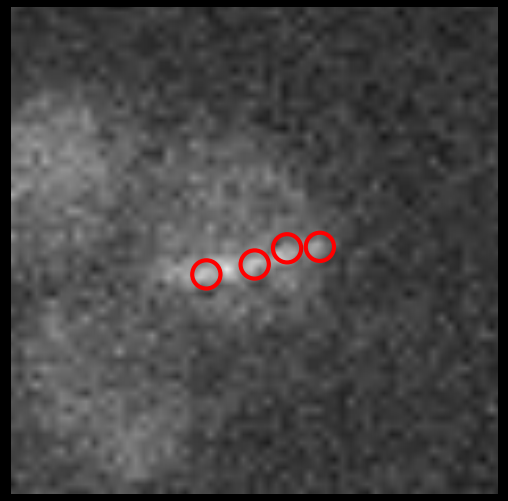


Our tools see dots. Biology made lines.

Raw image



Spot detector output



Current tool	What goes wrong	What we need
Spot detector	1 filament → 1–5 false spots	Filament classifier
No shape model	Cannot tell a spot from a rod	Elongation + length
No tracker	Filament lifetime unknown	Frame-to-frame tracking
Manual counting	1000s of cells, time consuming	Automated, reproducible

What you'll build: a Python filament module

Full pipeline

Raw image → **Filament Detector** → Length measurement → Frame tracking → Lifetime output

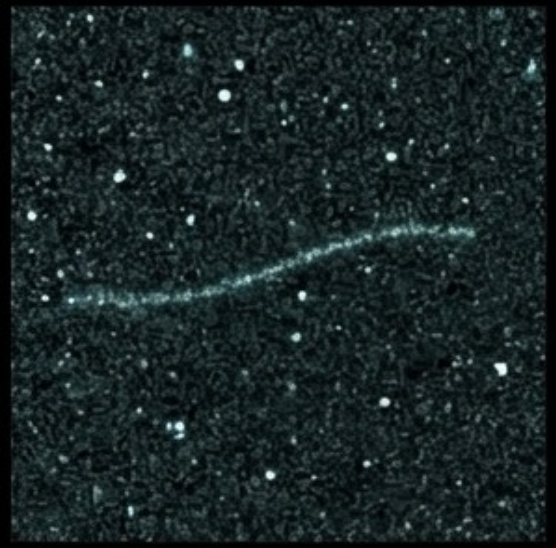
Core modules

1. Detect
Filaments vs. spots & noise

2. Measure
Length per cell per timepoint

3. Track
Formation & dissolution timing

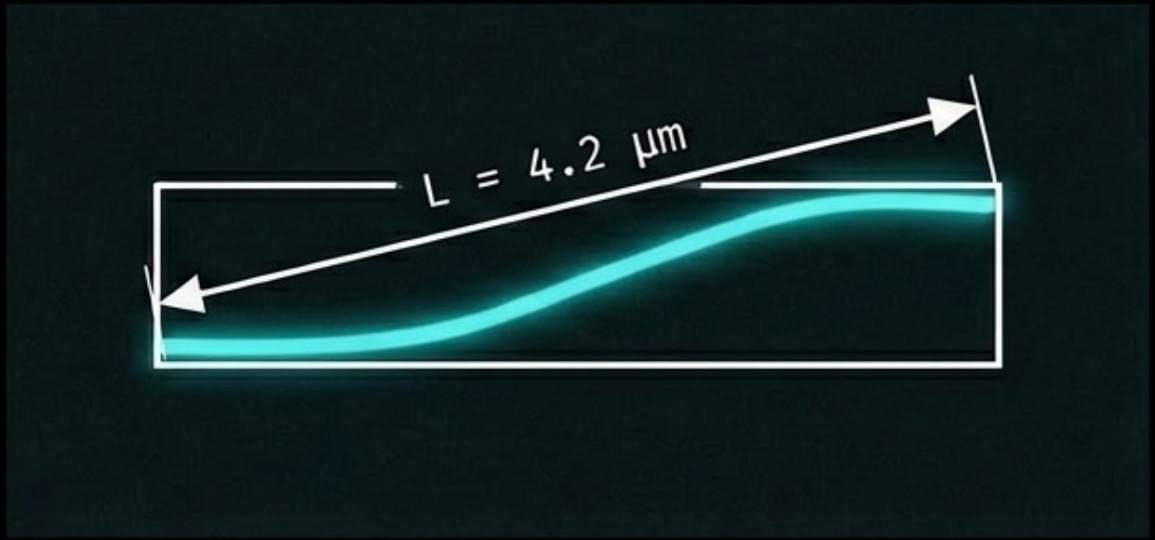
[RAW IMAGE] → **[MODULE 1: DETECT]** → [MODULE 2: MEASURE] → [MODULE 3: TRACK]



```
def detect_filaments(image):
    Objective: Reliably isolate
    filamentous structures from
    background noise and non-
    filamentous spots.

    Challenge: Must handle varying
    signal-to-noise ratios in
    fluorescence data.
```

[RAW IMAGE] → [MODULE 1: DETECT] → [MODULE 2: MEASURE] → [MODULE 3: TRACK]

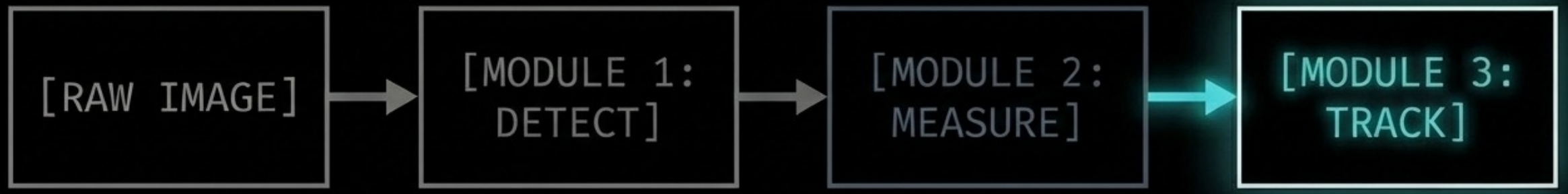


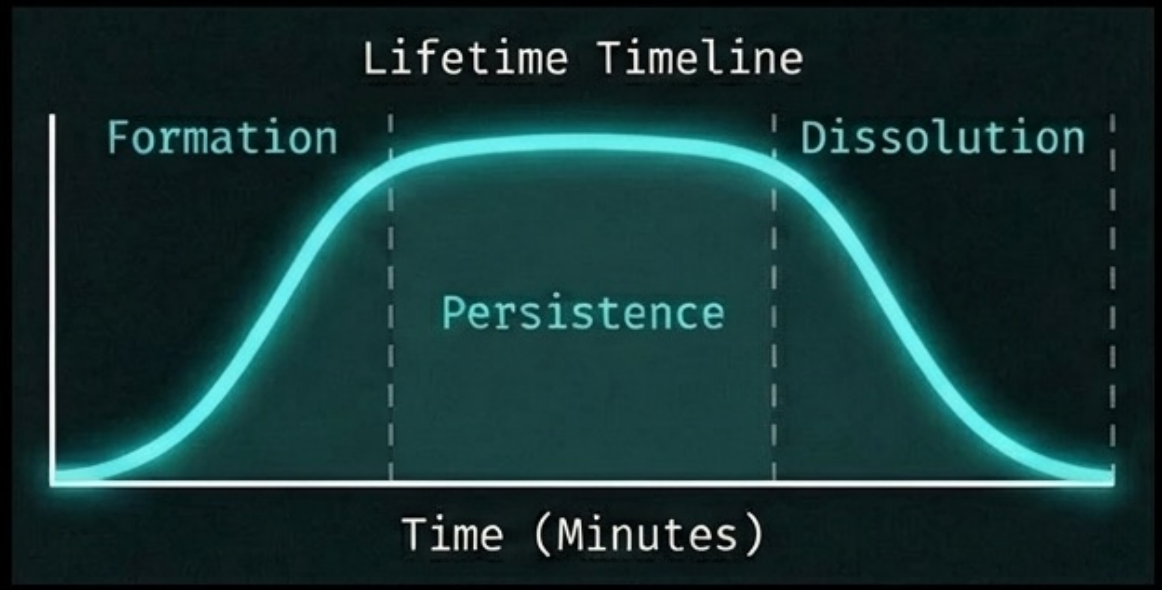

```
def extract_geometry(filaments):

    Objective: Extract precise
    geometric data for every
    detected object.

    Output Required: Filament
    length, per cell, per
    individual timepoint.
```

BONUS





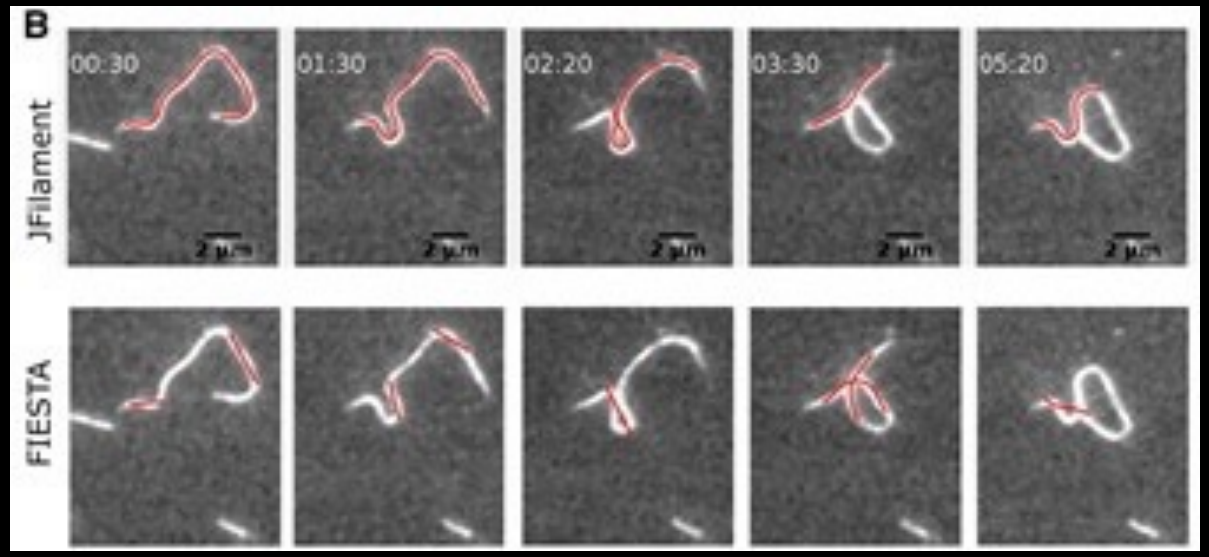

```
def track_lifecycle(frames):

    Objective: Link discrete
    filament detections across
    sequential frames.

    Output Required: Exact timing
    of when a filament appears and
    when it dissolves.
```

What you'll have from day one

- ✓ Real fluorescence datasets
- ✓ Existing algorithm to evaluate
- ✓ MATLAB codebase



Join the challenge!

Ideal skills:

Image processing · Computer vision
Python (scikit-image, NumPy, TrackPy)
Machine learning · Deep learning

Biology knowledge: not required.

Rewards:

1. Have fun with some cool science
2. Glory to contribute to new breakthroughs
3. Fitbit Inspire 3 tracker (no filament included)

What success looks like

detect_filaments(image)

→ filament_count

→ mean_length_px

→ time_of_appearance

Bonus: track across frames, measure dissolution timing